

GCP Data Engineer – Master Interview Preparation Guide

1. What are DAG services in Airflow and GCP?

Definition

A DAG (Directed Acyclic Graph) in Airflow represents a workflow where tasks are connected with dependencies and executed in a defined sequence.

In GCP, DAG orchestration is mainly handled using:

- Cloud Composer (Managed Airflow)
- Workflows
- Dataform scheduling
- Cloud Scheduler

Architecture Flow

Source → Airflow DAG → GCS → Dataflow/Dataproc → BigQuery → Reporting

Common DAG Components

- DAG
- Operators
- Sensors
- Tasks
- Task Dependencies
- Scheduler
- Executor
- XCom

Airflow Operators Used in Projects

- PythonOperator
- BashOperator
- BigQueryOperator
- GCSToBigQueryOperator
- BranchPythonOperator
- DummyOperator
- TriggerDagRunOperator

Example

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime
```

```
with DAG('sample_dag', start_date=datetime(2025,1,1),
schedule_interval='@daily') as dag:
```

```
def extract():  
    print('Extracting Data')  
  
task1 = PythonOperator(  
    task_id='extract_task',  
    python_callable=extract  
)
```

Real-Time Use in Project

In my project, Cloud Composer orchestrates ingestion from Oracle, lands data into GCS, triggers Dataflow jobs, loads curated data into BigQuery, and sends email alerts in case of failures.

Tricky Interview Questions

1. What happens if a DAG fails in the middle?
2. Difference between DAG and Task?
3. What is catchup in Airflow?
4. What is backfill?
5. Difference between Airflow and Cloud Workflows?

2. What types of transformations have you done?

Definition

Transformation means converting raw data into a clean and analytics-ready format.

Common Transformations

- Filtering
- Aggregation
- Joins
- Deduplication
- Null Handling
- Window Functions
- Pivot/Unpivot
- Flattening Nested JSON
- SCD Type 1 & Type 2
- Data Masking
- Schema Validation

PySpark Examples

```
from pyspark.sql.functions import *
```

```
df = df.filter(col('salary') > 50000)
```

Real-Time Use

I worked on transforming raw customer transaction data into curated sales marts using PySpark and BigQuery SQL.

Optimization Techniques Used

- Partitioning
- Broadcast Join
- Repartition
- Predicate Pushdown
- Caching

Tricky Questions

1. Difference between narrow and wide transformations?
2. What is shuffle?
3. What is data skew?
4. How did you optimize joins?
5. What transformations are expensive?

3. From which types of sources & systems are you getting data?

Common Data Sources

- Oracle
- MySQL
- SQL Server
- PostgreSQL
- APIs
- SAP
- Salesforce
- Flat Files (CSV, JSON, Parquet)
- Kafka
- Pub/Sub
- SFTP
- MongoDB

Batch Sources

- ERP systems
- Banking systems
- CRM systems

Streaming Sources

- Kafka

- Pub/Sub
- IoT streams

Real-Time Use

We ingest customer transaction data from Oracle using Sqoop/Dataflow and API-based marketing data using Python.

Tricky Questions

1. Difference between batch and streaming ingestion?
2. Which source creates schema evolution issues?
3. How do you ingest CDC data?
4. What if source schema changes suddenly?
5. Which format is best for analytics?

4. What is the volume of data?

Answer Strategy

Never give unrealistic numbers.

Sample Interview Answer

- Daily ingestion: 200 GB – 2 TB
- Monthly processing: 20–50 TB
- Streaming events: 1–5 million/day

Explain Properly

Mention:

- Batch volume
- Streaming volume
- Peak load
- Number of tables
- SLA timing

Real-Time Example

Our sales pipeline processes around 800 GB daily and near real-time Pub/Sub messages every minute.

Tricky Questions

1. How do you handle huge datasets?
2. What if data suddenly doubles?
3. Which service handled maximum load?
4. How do you reduce processing time?
5. What bottleneck did you face?

5. Python code to read CSV, transform, and move to BigQuery

Example

```
from google.cloud import bigquery
```

```

import pandas as pd

# Read CSV
_df = pd.read_csv('employees.csv')

# Transformation
_df['salary'] = _df['salary'].fillna(0)

# BigQuery Client
client = bigquery.Client()

table_id = 'project.dataset.employee'

job = client.load_table_from_dataframe(_df, table_id)
job.result()

print('Loaded Successfully')

```

Real-Time Use

Used Python ETL scripts to load API and CSV-based finance files into BigQuery.

Tricky Questions

1. Difference between pandas and PySpark?
2. Which method is best for huge files?
3. How do you handle schema mismatch?
4. How do you authenticate in GCP?
5. Difference between append and truncate?

6. SQL code for removing duplicate rows Using ROW_NUMBER()

```

SELECT *
FROM (
  SELECT *,
  ROW_NUMBER() OVER(
    PARTITION BY emp_id
    ORDER BY updated_date DESC
  ) rn
FROM employee_raw
)
WHERE rn = 1;

```

Explanation

- PARTITION BY groups duplicates
- ROW_NUMBER identifies latest record
- Keep rn=1 only

Real-Time Use

Used in bronze-to-silver layer data cleansing.

Alternative Methods

- DISTINCT
- GROUP BY
- QUALIFY

Tricky Questions

1. Difference between DISTINCT and ROW_NUMBER?
2. Why ROW_NUMBER preferred?
3. Difference between RANK and DENSE_RANK?
4. Can duplicates exist with NULLs?
5. Which dedup method is fastest?

7. Partitioning and Clustering in BigQuery

Partitioning

Definition

Splits table into smaller partitions based on date/timestamp/integer.

Syntax

```
CREATE TABLE sales
PARTITION BY DATE(order_date)
AS
SELECT * FROM source_table;
```

Benefits

- Reduces scanned data
- Improves performance
- Reduces cost

Clustering

Definition

Organizes data physically based on columns.

Syntax

```
CREATE TABLE sales
```

PARTITION BY DATE(order_date)
CLUSTER BY customer_id;

Real-Time Use

Partitioned by transaction_date and clustered by customer_id.

Tricky Questions

1. Max clustering columns?
2. Difference between partition and cluster?
3. Which is more cost effective?
4. Can string be partitioned?
5. What happens without partition filter?

8. What are policies in BigQuery?

Types of Policies

- IAM Policies
- Row-Level Security
- Column-Level Security
- Data Masking Policies

Row Level Security Example

```
CREATE ROW ACCESS POLICY finance_policy  
ON dataset.sales  
GRANT TO ('group:finance@company.com')  
FILTER USING (department='Finance');
```

Real-Time Use

Restricted salary columns for HR users only.

Tricky Questions

1. Difference between IAM and Row Access Policy?
2. Can users bypass policies?
3. What is authorized view?
4. What is dynamic masking?
5. How do you secure PII?

9. Explain your project

Sample Answer Structure

1. Project Overview
2. Data Sources
3. GCP Services Used

4. Data Flow
5. Transformations
6. Security
7. Optimization
8. Monitoring

Example

I worked on a banking analytics platform where data from Oracle, APIs, and Kafka was ingested into GCS using Dataflow. PySpark transformations were applied in Dataproc and final curated data was loaded into BigQuery for Power BI reporting.

Services Used

- GCS
- Composer
- Dataflow
- Dataproc
- BigQuery
- Pub/Sub
- IAM

Tricky Questions

1. Biggest challenge?
2. How much data?
3. SLA?
4. How many pipelines?
5. How did you optimize cost?

10. How to extract data from Oracle?

Methods

- Sqoop
- JDBC
- Dataflow JDBC Connector
- Python cx_Oracle
- Informatica

PySpark JDBC Example

```
jdbcDF = spark.read.format('jdbc') \  
.option('url', 'jdbc:oracle:thin:@host:1521:xe') \  
.option('dbtable', 'EMP') \  
.option('user', 'scott') \  
.option('password', 'tiger') \  
.load()
```


Real-Time Use

Incremental extraction using timestamp column.

Tricky Questions

1. How do you load incremental data?
2. What if Oracle table is huge?
3. What is fetch size?
4. How do you parallelize extraction?
5. What is CDC?

11. How to upload data into GCS?

Methods

- gsutil
- Python
- Dataflow
- Storage Transfer Service
- Console UI

Python Example

```
from google.cloud import storage
```

```
client = storage.Client()
bucket = client.bucket('demo-bucket')
blob = bucket.blob('emp.csv')
blob.upload_from_filename('emp.csv')
```

Tricky Questions

1. Difference between Nearline and Coldline?
2. Max file size?
3. What is lifecycle policy?
4. Difference between object and bucket?
5. How to secure bucket?

12. Cost Optimization Techniques in GCP & BigQuery

BigQuery Optimization

- Partition tables
- Clustering
- Avoid SELECT *
- Use materialized views

- Use approximate functions
- Filter partitions

Spark Optimization

- Broadcast Join
- Repartition
- Avoid shuffle
- Cache DataFrames

Storage Optimization

- Use Parquet/Avro
- Compress files
- Use correct storage class

Real-Time Use

Reduced query cost by 40% using partition pruning.

Tricky Questions

1. How to reduce scanned bytes?
2. Which format is cheaper?
3. Difference between on-demand and flat-rate?
4. Why avoid SELECT *?
5. How to monitor cost?

13. Day-to-day activities of GCP Data Engineer

Activities

- Monitor Airflow DAGs
- Develop ETL pipelines
- Optimize SQL queries
- Handle incidents
- Work with business teams
- Perform code deployments
- Validate data quality
- Handle schema changes

Tools Used

- BigQuery
- Composer
- Dataflow
- Dataproc
- Git
- Jenkins

Tricky Questions

1. Production support responsibilities?
2. How do you handle failures?

3. What is RCA?
4. How do you deploy code?
5. How do you validate data?

14. STRUCT Data Type in BigQuery

Definition

STRUCT stores nested records.

Example

```
SELECT customer.address.city  
FROM customer;
```

Creating STRUCT

```
SELECT STRUCT('Ravi' AS name, 25 AS age) AS emp;
```

Real-Time Use

Used nested JSON structures from APIs.

Tricky Questions

1. Difference between STRUCT and ARRAY?
2. What is UNNEST?
3. Can STRUCT contain ARRAY?
4. How do you flatten nested data?
5. Performance impact of nested data?

15. Airflow Operators

Common Operators

- PythonOperator
- BashOperator
- EmailOperator
- BigQueryOperator
- BranchPythonOperator
- TriggerDagRunOperator

Real-Time Use

Used PythonOperator for validations and BigQueryOperator for SQL execution.

Tricky Questions

1. Difference between Operator and Sensor?
2. What is BranchPythonOperator?

3. What is TriggerRule?
4. Difference between taskflow and operator?
5. How do retries work?

16. Airflow Dependency Questions

T1 runs first, then T2 & T3 parallelly

T1 >> [T2, T3]

T1 always runs, either T2 or T3 should run

Use BranchPythonOperator.

Tricky Questions

1. What is upstream/downstream?
2. What is branching?
3. How to trigger DAG from DAG?
4. What is dynamic DAG?
5. What is TaskGroup?

17. Task failing for 7 days – how to troubleshoot?

Steps

1. Check Airflow logs
2. Check upstream dependency
3. Validate source availability
4. Check schema changes
5. Validate permissions
6. Check resource utilization
7. Re-run failed task

Real-Time Example

A pipeline failed due to schema evolution in source CSV.

Tricky Questions

1. How do you retry failed jobs?
2. Difference between retry and rerun?
3. How do you debug Airflow?
4. What logs do you check?
5. How to alert failures?

18. XCom in Airflow

Definition

XCom is used for task-to-task communication.

Example

```
xcom_push(key='count', value=100)
```

Real-Time Use

Used to pass dynamic filenames between tasks.

Tricky Questions

1. Can XCom store huge data?
2. Where is XCom stored?
3. Alternatives to XCom?
4. Difference between Variables and XCom?
5. XCom limitations?

19. SparkSession Creation

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName('Demo') \
    .getOrCreate()
```

Tricky Questions

1. Difference between SparkContext and SparkSession?
2. What is driver?
3. What is executor?
4. What is lazy evaluation?
5. What creates DAG in Spark?

20. Lazy Evaluation in Spark

Definition

Transformations are not executed immediately. Execution happens only after action.

Example

```
df.filter(col('salary') > 5000)
```

No execution until:

```
df.show()
```

Benefits

- Optimization
- Reduced execution
- Better performance

Tricky Questions

1. What are actions?
2. Difference between transformation and action?
3. Why Spark is faster?
4. What is DAG optimization?
5. What is lineage?

21. Read JSON in PySpark

```
df = spark.read.json('emp.json')
```

Multiline JSON

```
df = spark.read.option('multiline','true').json('emp.json')
```

Tricky Questions

1. Difference between JSON and Parquet?
2. What is schema inference?
3. How to flatten nested JSON?
4. How to handle corrupt records?
5. Which format is faster?

22. Read CSV and Load into BigQuery using PySpark

```
df = spark.read.csv('emp.csv', header=True)
```

```
df.write.format('bigquery') \
.option('table','dataset.emp') \
.save()
```

Tricky Questions

1. Which connector used?
2. How to load incrementally?
3. How to optimize write?
4. Append vs overwrite?
5. What is intermediate GCS bucket?

23. Second Largest Number Without Built-in Functions

```
lst = [10,20,5,8,30]
```

```
largest = second = float('-inf')
```

```
for i in lst:  
    if i > largest:  
        second = largest  
        largest = i  
    elif i > second and i != largest:  
        second = i
```

```
print(second)
```

Tricky Questions

1. Handle duplicates?
2. Third largest?
3. Negative numbers?
4. Time complexity?
5. Without sorting?

24. Difference Between OLTP and OLAP

Feature	OLTP	OLAP
Purpose	Transactions	Analytics
Queries	Small	Complex
Updates	Frequent	Less
Example	Banking	Reporting

Tricky Questions

1. Why BigQuery is OLAP?
2. Why OLTP not suitable for analytics?

3. Star schema used where?
4. Which DB for transactions?
5. Columnar storage benefits?

25. Joins in SQL

INNER JOIN

Returns matching records.

LEFT JOIN

Returns all left records.

RIGHT JOIN

Returns all right records.

FULL JOIN

Returns all records.

CROSS JOIN

Cartesian product.

Tricky Questions

1. Difference between left and full join?
2. Which join creates huge data?
3. Self join use case?
4. Anti join?
5. Semi join?

26. Window Functions

Common Functions

- ROW_NUMBER
- RANK
- DENSE_RANK
- LEAD
- LAG
- SUM OVER

Example

```
SELECT emp_id,  
salary,  
ROW_NUMBER() OVER(ORDER BY salary DESC) rn  
FROM emp;
```

Tricky Questions

1. Difference between rank functions?

2. What is window frame?
3. Running total query?
4. Dedup using row_number?
5. Performance issues?

27. BigQuery Architecture

Components

- Storage Layer
- Dremel Query Engine
- Colossus Storage
- Jupiter Network
- Slots

Features

- Serverless
- Columnar Storage
- Massively Parallel Processing

Tricky Questions

1. What are slots?
2. What is Dremel?
3. Why BigQuery fast?
4. What is separation of storage and compute?
5. Difference from RDBMS?

28. Slots in BigQuery

Definition

Slots are computational units used to execute queries.

Types

- On-demand slots
- Reserved slots

Interview Answer

We mainly use on-demand slots; enterprise teams may use reservations.

Tricky Questions

1. Difference between reservation and on-demand?
2. What if slots exhausted?
3. Slot autoscaling?
4. How to monitor slot usage?
5. Why queries queue?

29. Native vs External Tables

Feature	Native	External
Storage	BQ Storage	GCS
Performance	Faster	Slower
Cost	Higher storage	Lower storage

Real-Time Use

External tables used for temporary exploration.

Tricky Questions

1. External table limitations?
2. Can external tables be partitioned?
3. Performance difference?
4. What if source deleted?
5. Security implications?

30. Views vs Materialized Views

Feature	View	Materialized View
Storage	No	Yes
Performance	Slower	Faster
Refresh	Real-time	Periodic

Tricky Questions

1. When to use MV?
2. MV limitations?
3. Can MV auto refresh?
4. Authorized views?
5. Cost implications?

31. GCS Storage Classes

Class	Usage
Standard	Frequent access
Nearline	Monthly access
Coldline	Quarterly access
Archive	Rare access

Real-Time Use

Archive used for compliance backup.

Tricky Questions

1. Retrieval charges?
2. Lifecycle rules?
3. Which is cheapest?

4. Which is fastest?
5. Can move automatically?

32. Cloud Composer

Definition

Managed Apache Airflow service in GCP.

Features

- Managed infrastructure
- Scheduling
- Monitoring
- Scalability

Real-Time Use

Used for enterprise ETL orchestration.

Tricky Questions

1. Composer vs Airflow?
2. Composer versions?
3. Backend database?
4. How scaling works?
5. How DAGs deployed?

33. Sensors in Airflow

Definition

Wait for external event.

Examples

- FileSensor
- ExternalTaskSensor
- GCSObjectExistenceSensor

Tricky Questions

1. Difference between sensor and operator?
2. Poke vs reschedule mode?
3. Sensor timeout?
4. Performance impact?
5. Smart sensors?

34. Difference Between Set and List

Feature	List	Set
Duplicates	Allowed	Not Allowed
Ordered	Yes	No
Mutable	Yes	Yes

Tricky Questions

1. Why set faster?
2. Can set contain list?
3. Hashing concept?
4. Immutable set?
5. Time complexity?

35. Incremental Data Load

Techniques

- Timestamp based
- CDC
- Watermark
- Merge logic

BigQuery MERGE

```
MERGE target t
USING source s
ON t.id=s.id
WHEN MATCHED THEN UPDATE SET t.name=s.name
WHEN NOT MATCHED THEN INSERT ROW;
```

Tricky Questions

1. Full vs incremental?
2. Watermark meaning?
3. Late arriving data?
4. CDC tools?
5. Duplicate handling?

36. SOX Compliance

Definition

Sarbanes-Oxley compliance ensures financial data security and auditing.

Key Points

- Access control
- Audit logging
- Segregation of duties
- Encryption

Real-Time Use

Restricted finance dataset access using IAM and row policies.

Tricky Questions

1. PII handling?

2. Audit logs?
3. Data masking?
4. Compliance monitoring?
5. Least privilege principle?

37. Pub/Sub

Definition

Messaging service for real-time streaming.

Components

- Topic
- Publisher
- Subscriber

Push vs Pull

Push → Pub/Sub sends data. Pull → Consumer fetches data.

Real-Time Architecture

Application → Pub/Sub → Dataflow → BigQuery

Tricky Questions

1. At least once delivery?
2. Ordering key?
3. Dead letter queue?
4. Retention?
5. Acknowledgement?

38. Dataflow vs Dataproc

Feature	Dataflow	Dataproc
Type	Serverless	Managed Cluster
Engine	Apache Beam	Spark/Hadoop
Scaling	Auto	Manual/Auto

Real-Time Use

Dataflow for streaming; Dataproc for Spark batch jobs.

Tricky Questions

1. Which is cheaper?
2. Which supports Hadoop?
3. Streaming support?
4. Autoscaling?
5. Cluster management?

39. SCD Type 1 & Type 2

Type 1

Overwrite old data.

Type 2

Maintain history.

Type 2 Columns

- start_date
- end_date
- is_active

Tricky Questions

1. Difference between SCD1 and SCD2?
2. Which type stores history?
3. Surrogate key?
4. Fact-dimension relationship?
5. Current flag usage?

40. QUALIFY in BigQuery

Definition

Filters window function output.

Example

```
SELECT *  
FROM emp  
QUALIFY ROW_NUMBER() OVER(PARTITION BY id ORDER BY salary  
DESC)=1;
```

Tricky Questions

1. Difference between WHERE and QUALIFY?
2. Why use QUALIFY?
3. Can QUALIFY use aggregate?
4. Order of execution?
5. Alternative without QUALIFY?

41. CTE vs Temp Table

Feature	CTE	Temp Table
Storage	No	Yes
Scope	Query	Session
Reusability	Low	High

Tricky Questions

1. Which performs better?
2. Recursive CTE?
3. Temp table indexing?

4. Memory usage?
5. Multiple references?

42. Difference Between UNION and UNION ALL

Feature	UNION	UNION ALL
Removes duplicates	Yes	No
Performance	Slower	Faster

Tricky Questions

1. Why UNION slower?
2. Which uses sorting?
3. Duplicate elimination cost?
4. Data type requirements?
5. Column count rules?

43. Rank vs Dense_Rank vs Row_Number

Function	Duplicate Rank	Skip Rank
ROW_NUMBER	No	No
RANK	Yes	Yes
DENSE_RANK	Yes	No

Tricky Questions

1. Dedup best function?
2. Top N queries?
3. Handling ties?
4. Ranking with partitions?
5. Real-time use?

44. Catchup in Airflow

Definition

Runs missed DAG executions.

Example

```
catchup=False
```

Tricky Questions

1. Backfill vs catchup?
2. Why disable catchup?
3. Production risks?
4. Scheduler load?

5. Manual rerun?

45. Data Skew

Definition

Uneven distribution of data across partitions.

Problems

- Slow tasks
- Long execution time
- Executor memory issues

Solutions

- Salting
- Repartition
- Broadcast Join

Tricky Questions

1. How identify skew?
2. Salting technique?
3. Skew in joins?
4. Adaptive execution?
5. Broadcast threshold?

46. Salting Technique

Definition

Add random key to distribute skewed data.

Example

```
from pyspark.sql.functions import concat, rand
```

Real-Time Use

Used for skewed customer IDs.

Tricky Questions

1. Why salting needed?
2. Salting disadvantages?
3. Random key range?
4. Broadcast vs salting?
5. Performance impact?

47. Fact and Dimension Tables

Fact Table

Contains measures.

Examples:

- sales_amount
- quantity

Dimension Table

Contains descriptive data.

Examples:

- customer
- product
- date

Tricky Questions

1. Grain meaning?
2. Surrogate key?
3. Slowly changing dimensions?
4. Star schema?
5. Snowflake schema?

48. Broadcast Join

Definition

Small table copied to all executors.

Example

```
from pyspark.sql.functions import broadcast
```

```
df.join(broadcast(dim_df), 'id')
```

Tricky Questions

1. When not to use broadcast?
2. Broadcast threshold?
3. Memory impact?
4. Shuffle avoidance?
5. Auto broadcast?

49. Data Modelling

Types

- Star Schema
- Snowflake Schema
- Fact Constellation

Real-Time Use

Designed sales mart using star schema.

Tricky Questions

1. Normalized vs denormalized?

2. Surrogate vs natural key?
3. Factless fact?
4. Slowly changing dimensions?
5. Degenerate dimensions?

50. Mutable vs Immutable in Python

Mutable

- list
- dict
- set

Immutable

- tuple
- string
- int

Tricky Questions

1. Why tuple faster?
2. Hashable objects?
3. Mutable default argument issue?
4. Can tuple contain list?
5. Memory optimization?